Project's Low-Level Design

AI Manufacturing Platform

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr no	Student Name	Enrolment Number
1.	Aaditya Shrivastava	EN22CS301007
2.	Aakansha Jaiswal	EN22CS301011
3.	Akanksha Sharma	EN22CS301080
4.	Akshita Bhawsar	EN22CS301098
5.	Abhishek Malviya	EN22CS301034

Group Name: Group 12D1

Project Number: DO-12

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-26

Table of Contents

1. Introduction
    a. Scope of the document
    b. Intended Audience
    c. System overview
2. System Design
    a. Application Design
    b. Process Flow
    c. Information Flow
    d. Components Design
    e. Key Design Considerations
    f. API Catalogue
3. References

1. Introduction

Scope of the document

This document describes the detailed low-level design of the AI Manufacturing Platform developed as part of the DevOps project. The system combines Generative AI and Agentic AI capabilities and is deployed using modern DevOps practices such as Docker, Kubernetes, and CI/CD pipelines.

The purpose of this document is to provide a clear understanding of how the system is structured internally, how different components interact and how the application is deployed and managed in a containerized environment. It also outlines the internal workflows, APIs, and design decisions taken during development.

Intended Audience

This document is intended for:

- Developers working on AI backend modules
- DevOps engineers responsible for deployment and automation
- Testers validating system behavior
- Faculty members and evaluators reviewing the project

System Overview

The AI Manufacturing Platform is designed to simplify and automate manufacturing-related tasks using AI.

The system allows a user to input a manufacturing idea in natural language and generates:

- A structured technical description
- A visual prototype image
- A detailed procurement or research report

The platform integrates two major systems:

- A **Generative AI module** that produces text and images
- A **Multi-Agent system** that performs research and generates professional reports

The application is built using Python and Streamlit and uses external AI services such as language models and image generation APIs. The system also maintains a local memory using a vector database to improve output quality and reuse past information. To ensure scalability and reliability, the application is containerized using Docker and deployed on Kubernetes with support for rolling updates and CI/CD automation.

2. System Design

Application Design

The system follows a modular and containerized architecture where different responsibilities are separated into logical components.
At a high level, the application consists of the following layers:

- **Frontend Layer**

The frontend is built using Streamlit and provides a simple and interactive interface. It allows users to enter prompts, view generated outputs and access previous results.

- **Backend Layer**

The backend handles all processing logic. It receives user requests, coordinates AI processing, and returns results. This layer is responsible for integrating the LLM, image generation model, and agent system.

- **AI Processing Layer**

This layer contains the core intelligence of the system. It includes:

- Language model for generating structured content
- Image generation model for creating visual outputs
- Multi-agent system for performing research and report generation

- **Storage Layer**

The system uses:

- ChromaDB for storing embeddings and enabling semantic search
- JSON-based storage for maintaining session memory and logs

From a deployment perspective, all components are packaged into Docker containers and managed using Kubernetes. The application runs inside services are used to expose it to users.

Process Flow

The system follows a sequential workflow from input to output.

1. The user enters a manufacturing-related query through the web interface.
2. The request is sent to the backend service.
3. The backend determines the type of processing required.
4. The request is passed to the AI layer where:
    - The image model generates a visual prototype
    - The agent system (if triggered) performs research and creates a report
5. The generated outputs are stored in the vector database for future reference.
6. The final response is sent back to the frontend and displayed to the user.

In the DevOps pipeline:

1. Code changes are pushed to the repository
2. A CI/CD pipeline builds a Docker image
3. The image is pushed to a container registry
4. Kubernetes pulls the new image and updates the deployment
5. Rolling updates ensure the application remains available during upgrades

Information Flow

The information flow in the system is designed to be simple and efficient.

- The user interacts with the frontend through a web interface
- The frontend sends HTTP requests to the backend
- The backend processes the request and communicates with:
  - AI APIs (for text and image generation)
  - Agent modules (for research tasks)
  - Storage systems (for saving and retrieving data)
- The response is then returned to the frontend

All internal communication happens using JSON over HTTP. External communication with AI services is done through secure API calls.

Component Design

- **The Backend Dockerfile (Multi-Stage Build)**

The backend is the central component of the system. It is responsible for handling user requests and coordinating all AI operations.

- Built using Python
- Integrates LLM APIs and agent logic
- Exposes REST APIs for communication

- **The Frontend Dockerfile**

The frontend is responsible for user interaction.

- Built using Streamlit
- Displays generated text, images, and reports
- Sends requests to backend APIs

- **AI Components**

Language Model Module

Generates structured manufacturing descriptions, technical details, and optimized prompts.

Image Generation Module

Produces visual representations of manufacturing concepts based on text input.

Agent System

Consists of multiple agents such as:

- Research Agent for gathering data
- Writer Agent for generating reports

- **Storage Components**
  - **ChromaDB** stores vector embeddings for semantic search
  - **JSON storage** maintains session data and logs
- **Kubernetes Components**
  - **Deployment** manages application instances and scaling
  - **Service** exposes the application to external users
  - **Pods** run the containerized application

Key Design Considerations (DevOps & AWS Deployment):

Several important decisions were made during the design of the system:

Scalability:

The use of Kubernetes allows the system to scale horizontally by increasing the number of pods based on demand.

High Availability:

Rolling updates are used during deployment to ensure that the application remains available without downtime.

Security:

Sensitive information such as API keys is not stored in code. Instead, it is managed using environment variables or secure configuration methods.

Portability:

The application is fully containerized, which allows it to run consistently across different environments such as local machines and cloud platforms.

Modularity:

The system is designed in a modular way so that components like AI models or agents can be replaced or upgraded independently.

Maintainability

Clear separation of concerns and structured APIs make the system easy to maintain and extend.

API Catalogue

The system exposes REST APIs for communication between frontend and backend.

Core APIs

Endpoint	Method	Description
/generate	POST	Generates text and image output
/agent/run	POST	Executes multi-agent workflow
/history	POST	Retrieves previous results

System APIs

Endpoint	Method	Description
/health	GET	Checks if the system is running
/version	GET	Returns application version

3. References

1. Docker Official Documentation
2. Kubernetes Official Documentation
3. OpenAI API Documentation
4. CrewAI Framework Documentation
5. Project reports developed during implementation.